

Minimal LLM Application Development with LangChain + OpenAI

Goal: learn the core ideas with the smallest useful examples.

We will build up in this order:

1. Prompt engineering
2. Structured outputs / JSON schemas
3. Tool calling
4. OpenAI vs Anthropic APIs
5. Failures, retries, validation, and hallucinations
6. One tiny application that connects everything

Keep the notebook simple. Run one section at a time.

0. Setup

Mental model

An LLM application is usually:

input → prompt → model → validated output → optional tools → final answer

LangChain gives us a common interface around model providers and application components.

For this notebook we use:

- `langchain-openai`
- `ChatOpenAI`
- Pydantic for validation
- LangChain tools

The API key must be stored as `OPENAI_API_KEY` in a `.env` file beside this notebook.
The setup fails immediately if the file or key is missing.

```
In [11]: # Run once if needed
%pip install -qU langchain==1.4.2 langchain-core==1.6.3 langchain-openai==1.
```

Note: you may need to restart the kernel to use updated packages.

```
In [12]: from pathlib import Path
from dotenv import dotenv_values, load_dotenv
```

```

dotenv_path = Path.cwd() / ".env"

if not dotenv_path.is_file():
    raise FileNotFoundError(
        f"Missing {dotenv_path}. Create it with OPENAI_API_KEY=your-key."
    )

env_values = dotenv_values(dotenv_path)
if not env_values.get("OPENAI_API_KEY"):
    raise RuntimeError(
        f"OPENAI_API_KEY is missing or empty in {dotenv_path}."
    )

load_dotenv(dotenv_path, override=True)

```

Out[12]: True

```

In [13]: from langchain_openai import ChatOpenAI

# Change the model here if needed.
MODEL = "gpt-5.6-luna"

llm = ChatOpenAI(
    model=MODEL,
    # Select the modern endpoint explicitly; avoid model-specific kwargs.
    use_responses_api=True,
)

response = llm.invoke("Say hello in one short sentence.")
print(response.content)

```

```

[{'type': 'text', 'text': 'Hello! How can I help you today?', 'annotations':
[], 'id': 'msg_071242d278948685006ab22e39e51c87d285b6ad9cd3e54540', 'phase':
'final_answer'}]

```

1. Prompt engineering

Theory

A prompt is the **instruction + context + input** given to the model.

A useful prompt usually makes four things clear:

- **Role/task** — what the model should do
- **Input** — what it should work on
- **Constraints** — rules it should follow
- **Output format** — what the answer should look like

Bad:

■ Tell me about this text.

Better:

Extract the main problem from this text. Use one sentence. Do not invent missing facts.

The goal is not to make prompts long. The goal is to remove ambiguity.

```
In [14]: text = "Our checkout page sometimes fails after users enter their card details"

prompt = f"""
Task: Identify the main problem.

Input:
{text}

Rules:
- Answer in one sentence.
- Do not invent information.
"""

result = llm.invoke(prompt)
print(result.content)
```

```
[{'type': 'text', 'text': 'The checkout page intermittently fails after users enter their card details.', 'annotations': [], 'id': 'msg_0a7130a1dd45d97d006ab22e3b197887d28256fac072635a8f', 'phase': 'final_answer'}]
```

Prompt templates

Hard-coded strings become messy when inputs change.

LangChain's `ChatPromptTemplate` separates the **fixed instructions** from the **variable input**.

```
In [15]: from langchain_core.prompts import ChatPromptTemplate

prompt_template = ChatPromptTemplate.from_messages([
    ("system", "You extract the main problem. Do not invent facts."),
    ("human", "Text: {text}\nAnswer in one sentence."),
])

chain = prompt_template | llm

result = chain.invoke({"text": text})
print(result.content)
```

```
[{'type': 'text', 'text': 'The checkout page intermittently fails after users enter their card details.', 'annotations': [], 'id': 'msg_07cedcaacb0cc042006ab22e3cf71c87d29d31f41f80b9a6f1', 'phase': 'final_answer'}]
```

Key idea

`prompt | llm` means:

format the prompt → send it to the model

This composition style is called a LangChain **Runnable chain**.

2. Structured outputs / JSON schemas

Theory

Normal LLM output is free-form text:

The priority is high and the problem is payment failure.
Applications often need predictable data:

```
{  
  "problem": "Payment failure",  
  "priority": "high"  
}
```

A schema defines what fields are allowed and what types they have.

With Pydantic we get:

- a schema
- parsing
- type validation
- clear Python objects

This is much safer than asking for JSON in the prompt and manually parsing it.

```
In [16]: from typing import Literal  
from pydantic import BaseModel, Field  
  
class Issue(BaseModel):  
    problem: str = Field(description="Short description of the problem")  
    priority: Literal["low", "medium", "high"]  
  
structured_llm = llm.with_structured_output(Issue)  
  
issue = structured_llm.invoke(  
    "Customers cannot complete payment during checkout."  
)  
  
issue
```

```
Out[16]: Issue(problem='Customers cannot complete payment during checkout.', priority='high')
```

```
In [17]: print(issue.problem)  
print(issue.priority)  
print(issue.model_dump())
```

Customers cannot complete payment during checkout.

high

```
{'problem': 'Customers cannot complete payment during checkout.', 'priority': 'high'}
```

Why this matters

Instead of trusting text like:

```
'{"priority": "probably very urgent"}'
```

our application expects the exact contract:

```
priority: "low" | "medium" | "high"
```

The LLM generates the data; **Pydantic defines the application contract.**

3. Tool calling

Theory

An LLM itself does not execute your Python functions.

Tool calling works like this:

1. We describe available tools to the model.
2. The model decides whether a tool is needed.
3. The model returns a **tool call** with arguments.
4. Our application executes the real function.
5. We send the tool result back to the model.
6. The model creates the final answer.

Important:

The model **requests** a tool call. Your code actually executes it.

```
In [34]: from langchain_core.tools import tool

@tool
def get_order_status(order_id: str) -> str:
    """Return the status of an order by order ID."""
    fake_database = {
        "A100": "shipped",
        "B200": "processing",
    }
    return fake_database.get(order_id, "order not found")

tools = [get_order_status]
llm_with_tools = llm.bind_tools(tools)
```

```
In [48]: ai_message = llm_with_tools.invoke(
        "What is the status of order A100?"
    )

ai_message
```

```
Out[48]: AIMessage(content=[{'arguments': '{"order_id": "A100"}', 'call_id': 'call_v2ySikGtMpbjMizvKAG21Idl', 'name': 'get_order_status', 'type': 'function_call', 'id': 'fc_0bdf259f04c863fb006ab2f374ac5087d28768e5748eefb0da', 'status': 'completed'}], additional_kwargs={}, response_metadata={'id': 'resp_0bdf259f04c863fb006ab2f374361c87d299d4a5a4df8809f2', 'created_at': 1790112628.0, 'metadata': {}, 'model': 'gpt-5.6-luna', 'object': 'response', 'service_tier': 'default', 'status': 'completed', 'model_provider': 'openai', 'model_name': 'gpt-5.6-luna'}, id='resp_0bdf259f04c863fb006ab2f374361c87d299d4a5a4df8809f2', tool_calls=[{'name': 'get_order_status', 'args': {'order_id': 'A100'}, 'id': 'call_v2ySikGtMpbjMizvKAG21Idl', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 57, 'output_tokens': 21, 'total_tokens': 78, 'input_token_details': {'cache_creation': 0, 'cache_read': 0}, 'output_token_details': {'reasoning': 0}})
```

The model has not executed `get_order_status`.

It has only produced something conceptually like:

```
{
  "name": "get_order_status",
  "args": {"order_id": "A100"}
}
```

Now **our Python code** executes it.

```
In [39]: tool_map = {tool.name: tool for tool in tools}
print(f"{tool_map=}")
tool_call = ai_message.tool_calls[0]
print(f"{tool_call=}")
selected_tool = tool_map[tool_call["name"]]
print(f"{selected_tool=}")
tool_result = selected_tool.invoke(tool_call["args"])
print(f"{tool_result=}")
```

```
tool_map={'get_order_status': StructuredTool(name='get_order_status', description='Return the status of an order by order ID.', args_schema=<class 'langchain_core.utils.pydantic.get_order_status'>, func=<function get_order_status at 0x113edad40>)}
```

```
tool_call={'name': 'get_order_status', 'args': {'order_id': 'A100'}, 'id': 'call_3ypDhfMQp4A490ljoEBvAGXw', 'type': 'tool_call'}
```

```
selected_tool=StructuredTool(name='get_order_status', description='Return the status of an order by order ID.', args_schema=<class 'langchain_core.utils.pydantic.get_order_status'>, func=<function get_order_status at 0x113edad40>)
```

```
tool_result='shipped'
```

Now send both the model's tool request and the real tool result back to the model.

```
In [40]: from langchain_core.messages import HumanMessage, ToolMessage
```

```

messages = [
    HumanMessage("What is the status of order A100?"),
    ai_message,
    ToolMessage(
        content=str(tool_result),
        tool_call_id=tool_call["id"],
    ),
]

final_response = llm_with_tools.invoke(messages)
print(final_response.content)

```

```

[{'type': 'text', 'text': 'Order A100 has shipped.', 'annotations': [], 'id': 'msg_0a86533b643c3de5006ab2f0f8743887d2bdf7e881e755bbb1', 'phase': 'final_answer'}]

```

Learning nugget: message types

LangChain represents each part of a conversation with a message type:

- **SystemMessage** : sets the model's overall role, rules, or behavior.
- **HumanMessage** : contains input from the user.
- **AIMessage** : contains the model's response. It can also contain tool calls, as `ai_message` does above.
- **ToolMessage** : contains the result returned by a tool. Its `tool_call_id` connects the result to the exact tool call that requested it.

The exchange above therefore follows this sequence:

HumanMessage → AIMessage with a tool call → ToolMessage with the result → final AIMessage

Keeping these roles explicit helps the model understand who produced each piece of information and how tool results fit into the conversation.

3.1 Automatic tool execution with an agent

The previous cells implemented the tool-calling protocol manually so that every step was visible. In a larger application, repeating that code for every possible tool call quickly becomes cumbersome.

A LangChain agent manages the same loop for us:

1. Send the conversation and tool definitions to the model.
2. If the model requests a tool, execute it.
3. Add the resulting **ToolMessage** to the conversation.
4. Call the model again with the updated conversation.
5. Repeat until the model returns a final answer with no more tool calls.

`create_agent` handles this loop, including requests that require multiple tool calls.

```
In [49]: from langchain.agents import create_agent

agent = create_agent(
    model=llm,
    tools=tools,
    system_prompt="You are a helpful order support assistant.",
)

agent_result = agent.invoke(
    {"messages": [HumanMessage("What is the status of order A100?")]}
)

final_message = agent_result["messages"][-1]
print(final_message.text)
```

Order **A100** has been **shipped**.

Adding human approval to an agent

Some tools change data or trigger real-world actions. For example, cancelling an order should usually require confirmation before execution. LangChain's human-in-the-loop middleware can pause the agent immediately before a configured tool runs.

The checkpointer saves the paused run, while the `thread_id` identifies the conversation that must be resumed. This example uses in-memory storage for learning; a production application should use persistent storage.

```
In [ ]: from langchain.agents.middleware import HumanInTheLoopMiddleware
        from langgraph.checkpoint.memory import InMemorySaver

@tool
def cancel_order(order_id: str) -> str:
    """Cancel an order by its ID."""
    # This is only a simulated side effect for the lesson.
    return f"Order {order_id} was cancelled."

approval_agent = create_agent(
    model=llm,
    tools=[cancel_order],
    middleware=[
        HumanInTheLoopMiddleware(
            interrupt_on={
                "cancel_order": {
                    "allowed_decisions": ["approve", "reject"]
                }
            }
        )
    ],
    checkpointer=InMemorySaver(),
)
```



```

approval_config = {"configurable": {"thread_id": "approval-demo-1"}}

pending = approval_agent.invoke(
    {"messages": [HumanMessage("Cancel order A100.")]},
    config=approval_config,
    version="v2",
)

# The tool has not run yet. Show the action awaiting review.
print(pending.interrupts)

```

The application would show this pending action to a reviewer. After the reviewer decides, resume the **same thread** with either `approve` or `reject`. In this notebook, changing the value below represents that human decision.

```

In [ ]: from langgraph.types import Command

decision = "approve" # Change to "reject" to deny the tool call.

resumed = approval_agent.invoke(
    Command(resume={"decisions": [{"type": decision}]}),
    config=approval_config,
    version="v2",
)

print(resumed.value["messages"][-1].text)

```

Which approach should you choose?

- Use `create_agent(model=llm, tools=tools)` when you want LangChain to run the model-tool loop automatically. This is a convenient default for assistants that may choose and combine tools dynamically.
- Use `llm.bind_tools(tools)` with your own loop when your application must control each step—for example, to validate arguments, request approval before a sensitive action, restrict when a tool can run, or implement custom error handling.

In both cases, the model only **requests** a tool call. Python code still executes the real function; the difference is whether your code manages that process directly or delegates it to the agent runtime.

Human approval can also be implemented in the manual approach: inspect the model's requested tool call, ask the user or reviewer for a decision, and invoke the selected tool only after approval. That offers maximum control, but your application must manage the paused state and continuation itself.

4. OpenAI vs Anthropic APIs

Theory

OpenAI and Anthropic are different model providers, but the application pattern is similar:

messages → model → response

Both support concepts such as:

- system/user messages
- structured data
- tool/function calling
- streaming
- token limits
- provider errors and rate limits

Without LangChain, provider-specific SDK code differs.

With LangChain, the interface is intentionally similar:

```
# OpenAI
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="...")

# Anthropic
from langchain_anthropic import ChatAnthropic
model = ChatAnthropic(model="...")
```

Then much of the application can continue using:

```
model.invoke(...)
model.with_structured_output(...)
model.bind_tools(...)
```

Main architectural lesson

Keep provider-specific configuration near the model initialization.

Keep business logic outside it.

That makes changing providers much easier.

This notebook stays on OpenAI to keep the learning project minimal.

5. Failures, retries, validation, hallucinations

These are different problems.

Problem	Meaning	Typical protection
API failure	timeout, rate limit, temporary server error	retry
Invalid structure	output does not match expected shape	schema validation
Bad tool input	model requests invalid arguments	validate tool schema
Hallucination	plausible but unsupported information	grounding + explicit uncertainty
Business-rule failure	technically valid but unacceptable data	application validation

Do not treat all failures as "prompt problems."

5.1 Retries

Retries are appropriate for **temporary technical failures**.

LangChain Runnables can be wrapped with retry behavior.

Keep retries bounded: infinite retries hide problems and waste money.

```
In [22]: reliable_llm = llm.with_retry(
        stop_after_attempt=3,
    )

result = reliable_llm.invoke("Return only the word: ready")
print(result.content)

[{'type': 'text', 'text': 'ready', 'annotations': [], 'id': 'msg_0713a9a76e01c215006ab22e427ab487d2b582eea52ef60407', 'phase': 'final_answer'}]
```

5.2 Validation

Structured output protects the boundary between the LLM and your application.

But schema validation alone is not always enough.

Example: an integer may be valid Python, while a negative quantity may still violate your business rules.

```
In [23]: from pydantic import BaseModel, Field

class OrderRequest(BaseModel):
    product: str
    quantity: int = Field(gt=0, le=100)

order_llm = llm.with_structured_output(OrderRequest)
```

```

order = order_llm.invoke(
    "I want 3 keyboards."
)

order

```

Out [23]: OrderRequest(product='keyboard', quantity=3)

5.3 Hallucinations

A hallucination is not necessarily malformed output.

This is valid JSON:

```
{"order_status": "delivered"}
```

But it is still wrong if no reliable source said the order was delivered.

Minimal strategy

For factual application data:

1. Get facts from a trusted source/tool.
2. Give those facts to the model.
3. Tell the model not to invent missing information.
4. Prefer an explicit `unknown` state when evidence is missing.

Validation checks shape. Grounding checks truth against evidence.

```

In [24]: class GroundedAnswer(BaseModel):
        answer: str
        supported: bool = Field(
            description="True only when the answer is directly supported by the
        )

        grounded_llm = llm.with_structured_output(GroundedAnswer)

        context = "Order A100 status: shipped."

        answer = grounded_llm.invoke(
            f"""
            Use ONLY the context below.

            Context:
            {context}

            Question:
            Has order A100 been delivered?

            If the context does not prove it, say that it is not known.
            """
        )

```

```
answer
```

```
Out[24]: GroundedAnswer(answer='It is not known whether order A100 has been delivered. The context only says that it has been shipped.', supported=True)
```

6. Final mini project

Customer-support order assistant

We now connect the concepts:

user question → prompt instructions

→ tool calling

→ real order data

→ structured final answer

→ validation

→ retry wrapper

The application has one trusted source: `get_order_status`.

The model should not invent an order status.

```
In [25]: from typing import Literal
from pydantic import BaseModel, Field

class SupportAnswer(BaseModel):
    answer: str
    status: Literal["shipped", "processing", "not_found", "unknown"]
    grounded: bool = Field(
        description="True only when the status comes from the tool result"
    )
```

```
In [26]: support_prompt = ChatPromptTemplate.from_messages([
    (
        "system",
        """
You are a minimal order-support assistant.

Rules:
- Use the order-status tool when an order status is requested.
- Never invent an order status.
- Base the final answer only on the tool result.
- If reliable information is unavailable, use status='unknown'.
""",
    ),
    ("human", "{question}"),
])

support_model = llm.bind_tools([get_order_status]).with_retry(
```

```
    stop_after_attempt=3
)
```

Step A — ask the model whether it needs a tool

```
In [27]: question = "Where is order B200?"

messages = support_prompt.invoke({"question": question}).to_messages()
first_response = support_model.invoke(messages)

first_response.tool_calls
```

```
Out[27]: [{'name': 'get_order_status',
  'args': {'order_id': 'B200'},
  'id': 'call_wmcAUbJ1JlEnev3NNmpIog15',
  'type': 'tool_call'}]
```

Step B — execute requested tools

In a larger application this becomes a loop. Here we keep it explicit so the mechanism stays visible.

```
In [28]: messages.append(first_response)

for call in first_response.tool_calls:
    selected_tool = tool_map[call["name"]]
    result = selected_tool.invoke(call["args"])

    messages.append(
        ToolMessage(
            content=str(result),
            tool_call_id=call["id"],
        )
    )

messages[-1]
```

```
Out[28]: ToolMessage(content='processing', tool_call_id='call_wmcAUbJ1JlEnev3NNmpIog15')
```

Step C — produce a validated final answer

```
In [29]: final_structured_model = llm.with_structured_output(SupportAnswer).with_retr
    stop_after_attempt=3
)

# Add one final instruction because this second model call must return our s
messages.append(
    HumanMessage(
        """
Return the final answer using the required structured schema.
Use only the tool result already present in the conversation.
"""
    )
)
```

```

Map 'order not found' to status='not_found'.
"""
    )
)

final_answer = final_structured_model.invoke(messages)
final_answer

```

Out[29]: SupportAnswer(answer='Order B200 is processing.', status='processing', grounded=True)

7. Put it into one small function

Now that every part is visible, we can hide the plumbing behind one function.

```

In [30]: def ask_order_assistant(question: str) -> SupportAnswer:
    messages = support_prompt.invoke({"question": question}).to_messages()

    # 1. Let model request tools
    first_response = support_model.invoke(messages)
    messages.append(first_response)

    # 2. Execute requested tools
    for call in first_response.tool_calls:
        tool = tool_map.get(call["name"])

        if tool is None:
            messages.append(
                ToolMessage(
                    content="Tool unavailable",
                    tool_call_id=call["id"],
                )
            )
            continue

        try:
            result = tool.invoke(call["args"])
        except Exception as exc:
            result = f"Tool failed: {type(exc).__name__}"

        messages.append(
            ToolMessage(
                content=str(result),
                tool_call_id=call["id"],
            )
        )

    # 3. Ask for final validated output
    messages.append(
        HumanMessage(
            """
Return the final structured answer.
Use only verified tool results.
If no verified result exists, use status='unknown' and grounded=False.
"""

```

```

Map 'order not found' to status='not_found'.
.....

    )
)

return final_structured_model.invoke(messages)

```

```
In [31]: ask_order_assistant("What is happening with order A100?")
```

```
Out[31]: SupportAnswer(answer='Order A100 has shipped.', status='shipped', grounded=True)
```

```
In [32]: ask_order_assistant("What is happening with order XYZ?")
```

```
Out[32]: SupportAnswer(answer='Order XYZ was not found.', status='not_found', grounded=True)
```

8. What you should be able to explain in an interview

Prompt engineering

A prompt defines the task, context, constraints, and expected output. Good prompting reduces ambiguity rather than simply adding more text.

Structured outputs

Schemas turn free-form model text into predictable application data. Pydantic gives us parsing and validation.

Tool calling

The model does **not** execute functions. It requests a tool call with structured arguments; application code executes the function and returns the result to the model.

OpenAI / Anthropic APIs

They are different providers with different SDKs, but both expose similar LLM concepts. LangChain gives the application a more consistent model interface.

Retries

Retry temporary infrastructure/API failures, with a bounded number of attempts.

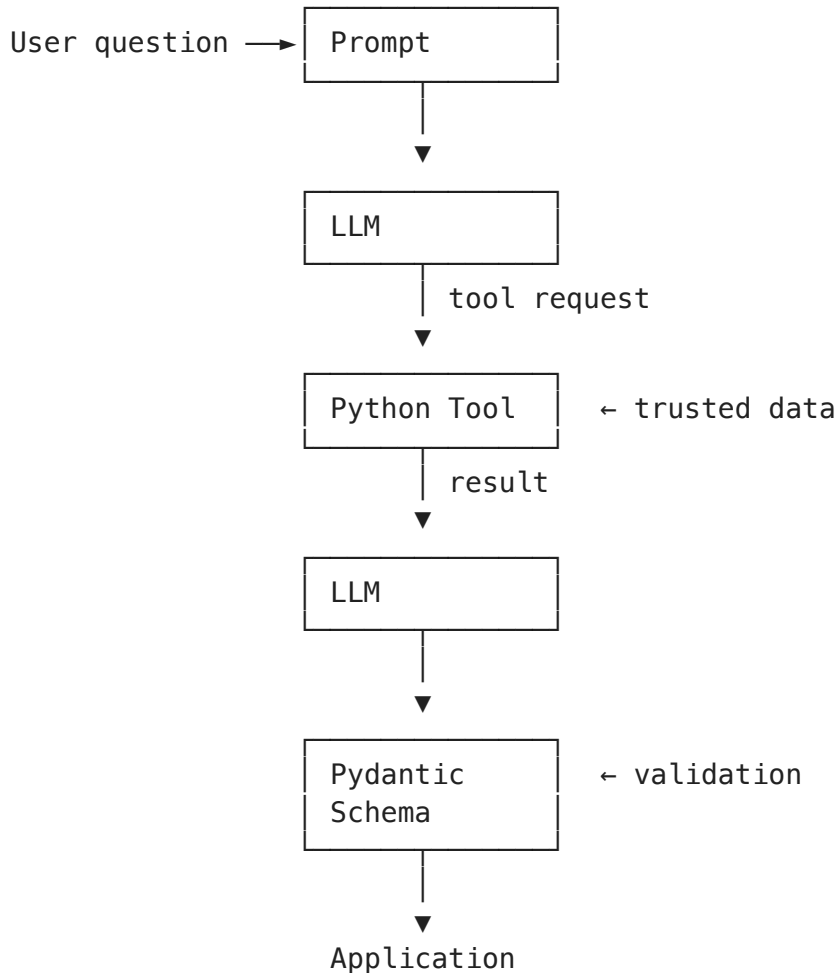
Validation

Validate model output before letting it enter business logic.

Hallucinations

A syntactically valid answer can still be factually wrong. Ground factual answers in trusted data/tools and allow the model to say that information is unknown.

9. Architecture recap



That is the core of many production LLM applications.

Once this is comfortable, the natural next topics are **conversation state, RAG, agents/LangGraph, tracing, and evaluation.**

10. Complete code overview

This final cell brings the required application code together in one place. It includes configuration, the schema, tool, prompt, retry behavior, tool execution, validation, and example calls.

Run the installation cell from section 0 first if the dependencies are not installed.

```
In [33]: from pathlib import Path
from typing import Literal

from dotenv import dotenv_values, load_dotenv
from langchain_core.messages import HumanMessage, ToolMessage
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field

# 1. Configuration: require OPENAI_API_KEY from .env
dotenv_path = Path.cwd() / ".env"
if not dotenv_path.is_file():
    raise FileNotFoundError(
        f"Missing {dotenv_path}. Create it with OPENAI_API_KEY=your-key."
    )

env_values = dotenv_values(dotenv_path)
if not env_values.get("OPENAI_API_KEY"):
    raise RuntimeError(
        f"OPENAI_API_KEY is missing or empty in {dotenv_path}."
    )

load_dotenv(dotenv_path, override=True)

MODEL = "gpt-5.6-luna"
llm = ChatOpenAI(
    model=MODEL,
    use_responses_api=True,
)

# 2. Validated application output
class SupportAnswer(BaseModel):
    answer: str
    status: Literal["shipped", "processing", "not_found", "unknown"]
    grounded: bool = Field(
        description="True only when the status comes from the tool result"
    )

# 3. Trusted tool
@tool
def get_order_status(order_id: str) -> str:
    """Return the status of an order by order ID."""
    fake_database = {
        "A100": "shipped",
        "B200": "processing",
    }
    return fake_database.get(order_id, "order not found")
```

4. Prompt and model wrappers

```
support_prompt = ChatPromptTemplate.from_messages([
    (
        "system",
        """
You are a minimal order-support assistant.
Use the order-status tool whenever an order status is requested.
Never invent a status. Base the final answer only on verified tool results.
If reliable information is unavailable, use status='unknown'.
""",
    ),
    ("human", "{question}"),
])

tools = [get_order_status]
tool_map = {available_tool.name: available_tool for available_tool in tools}
support_model = llm.bind_tools(tools).with_retry(stop_after_attempt=3)
final_model = llm.with_structured_output(SupportAnswer).with_retry(
    stop_after_attempt=3
)
```

5. Complete application flow

```
def ask_order_assistant(question: str) -> SupportAnswer:
    messages = support_prompt.invoke({"question": question}).to_messages()

    # Ask the model which tools it needs.
    first_response = support_model.invoke(messages)
    messages.append(first_response)

    # Execute every requested tool in application code.
    for call in first_response.tool_calls:
        selected_tool = tool_map.get(call["name"])

        if selected_tool is None:
            result = "Tool unavailable"
        else:
            try:
                result = selected_tool.invoke(call["args"])
            except Exception as exc:
                result = f"Tool failed: {type(exc).__name__}"

        messages.append(
            ToolMessage(
                content=str(result),
                tool_call_id=call["id"],
            )
        )

    # Convert the grounded result into the validated schema.
    messages.append(
        HumanMessage(
            """
```

Return the final structured answer using only verified tool results.
If there is no verified result, use status='unknown' and grounded=False.

```

Map 'order not found' to status='not_found'.
"""

    )
)

return final_model.invoke(messages)

# 6. Examples
print(ask_order_assistant("What is happening with order A100?"))
print(ask_order_assistant("What is happening with order XYZ?"))

```

```

answer='Order A100 has shipped.' status='shipped' grounded=True
answer='Order XYZ was not found.' status='not_found' grounded=True

```

10.1 Complete code using `create_agent`

The manual version above exposes each step. The version below builds the same order assistant with an agent, which owns the repeated **model** → **tool** → **model** loop.

It is still our responsibility to define the trusted tool, the behavioral rules, and the validated output schema. The agent removes the orchestration code that binds tools, dispatches calls, creates `ToolMessage` objects, and decides when another model call is needed.

```

In [50]: from pathlib import Path
from typing import Literal

from dotenv import dotenv_values, load_dotenv
from langchain.agents import create_agent
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field

# 1. Configuration: require OPENAI_API_KEY from .env
dotenv_path = Path.cwd() / ".env"
if not dotenv_path.is_file():
    raise FileNotFoundError(
        f"Missing {dotenv_path}. Create it with OPENAI_API_KEY=your-key."
    )

env_values = dotenv_values(dotenv_path)
if not env_values.get("OPENAI_API_KEY"):
    raise RuntimeError(
        f"OPENAI_API_KEY is missing or empty in {dotenv_path}."
    )

load_dotenv(dotenv_path, override=True)

MODEL = "gpt-5.6-luna"
llm = ChatOpenAI(
    model=MODEL,

```

```

    use_responses_api=True,
    max_retries=2,
)

# 2. Validated application output
class SupportAnswer(BaseModel):
    answer: str
    status: Literal["shipped", "processing", "not_found", "unknown"]
    grounded: bool = Field(
        description="True only when the status comes from the tool result"
    )

# 3. Trusted tool
@tool
def get_order_status(order_id: str) -> str:
    """Return the status of an order by order ID."""
    fake_database = {
        "A100": "shipped",
        "B200": "processing",
    }
    return fake_database.get(order_id, "order not found")

# 4. Agent: tool loop and structured output are configured together
support_agent = create_agent(
    model=llm,
    tools=[get_order_status],
    response_format=SupportAnswer,
    system_prompt="""
You are a minimal order-support assistant.
Use the order-status tool whenever an order status is requested.
Never invent a status. Base the final answer only on verified tool results.
Map 'order not found' to status='not_found'.
If no verified tool result is available, use status='unknown' and grounded=False
""",
)

# 5. Complete application flow
def ask_order_agent(question: str) -> SupportAnswer:
    result = support_agent.invoke(
        {"messages": [{"role": "user", "content": question}]}
    )
    return result["structured_response"]

# 6. Examples
print(ask_order_agent("What is happening with order A100?"))
print(ask_order_agent("What is happening with order XYZ?"))

```

```

answer='Order A100 has shipped.' status='shipped' grounded=True
answer='Order XYZ was not found.' status='not_found' grounded=True

```

Manual flow versus create_agent

Concern	Manual implementation	create_agent implementation
Expose tools to the model	Explicit <code>llm.bind_tools(tools)</code>	Pass <code>tools</code> to <code>create_agent</code>
Execute tool requests	Look up and invoke each tool yourself	Agent dispatches the requested tool
Return tool results	Construct each <code>ToolMessage</code> yourself	Agent adds tool results to its state
Multiple rounds	Write and control the loop yourself	Agent repeats until no more tool calls remain
Structured result	Make a separate structured-model call	Pass <code>response_format=SupportAnswer</code> and read <code>structured_response</code>
Control	Maximum control over every step	Less plumbing; customize through prompts, middleware, and agent configuration

Choose the manual flow for tightly controlled or deterministic workflows where every transition must be explicit. Choose `create_agent` when the model may dynamically select tools or require several tool rounds and the standard agent loop fits the application.

In []: